
--- MATH135A Project 2: Problem 1 ---

Solves a 4x4 system using Gaussian elimination with scaled partial pivoting

```
% --- 1. Setup ---
A = [ 0.4096, 0.1234, 0.3678, 0.2943;
      0.2246, 0.3872, 0.4015, 0.1129;
      0.3645, 0.1920, 0.3781, 0.0643;
      0.1784, 0.4002, 0.2786, 0.3927];

B = [ 0.4042; % Using the value 0.4042 as you provided
      0.1250;
      0.4255;
      0.2557];

% Get the size of the matrix
n = 4;

% Create the augmented matrix
Aug = [A, B];

% Create the scale vector (as a column)
% We can also calculate this in MATLAB:
s = max(abs(A), [], 2); % Finds max absolute value in each row

disp('Initial Augmented Matrix:');
disp(Aug);
disp('Initial Scale Vector:');
disp(s);

% --- 2. Forward Elimination ---
for k = 1:n-1

    % --- Pivoting ---
    % Find scaled ratios for rows k through n in column k
    ratios = abs(Aug(k:n, k)) ./ s(k:n);

    % Find the index of the largest ratio
    [max_val, max_idx] = max(ratios);

    % Calculate the actual row number to pivot with
    pivot_row = k + max_idx - 1;

    % Swap rows if pivot_row is different from the current row k
    if pivot_row ~= k
        % Swap rows in Aug matrix
        temp_row = Aug(k, :);
        Aug(k, :) = Aug(pivot_row, :);
        Aug(pivot_row, :) = temp_row;

        % IMPORTANT: Swap elements in the scale vector too
        temp_s = s(k);
```

```

        s(k) = s(pivot_row);
        s(pivot_row) = temp_s;
    end

    % --- Elimination ---
    % Loop through all rows below the pivot row k
    for i = k+1:n

        % Find the multiplier
        m = Aug(i, k) / Aug(k, k);

        % Subtract the scaled pivot row from the current row i
        Aug(i, :) = Aug(i, :) - m * Aug(k, :);
    end
end

disp('Upper Triangular (Eliminated) Matrix:');
disp(Aug);

% --- 3. Back Substitution ---
% Create an empty solution vector (filled with zeros)
x = zeros(n, 1);

% Solve for the last element, x(n)
x(n) = Aug(n, n+1) / Aug(n, n);

% Loop backwards from n-1 down to 1
for k = n-1:-1:1

    % Calculate the sum of the parts we already know
    % (Aug(k, k+1:n) is a row vector, x(k+1:n) is a column vector)
    sum_of_known_parts = Aug(k, k+1:n) * x(k+1:n);

    % Solve for x(k)
    x(k) = (Aug(k, n+1) - sum_of_known_parts) / Aug(k, k);
end

% --- 4. Final Answer ---
disp('Final Answer Vector x:');
disp(x);

Initial Augmented Matrix:
    0.4096    0.1234    0.3678    0.2943    0.4042
    0.2246    0.3872    0.4015    0.1129    0.1250
    0.3645    0.1920    0.3781    0.0643    0.4255
    0.1784    0.4002    0.2786    0.3927    0.2557

Initial Scale Vector:
    0.4096
    0.4015
    0.3781
    0.4002

```

Upper Triangular (Eliminated) Matrix:

0.4096	0.1234	0.3678	0.2943	0.4042
0	0.3465	0.1184	0.2645	0.0797
-0.0000	0	0.0906	-0.2924	-0.1701
0.0000	0	0	-0.1871	0.0895

Final Answer Vector x:

3.8718
1.7649
-3.4220
-0.4787

Published with MATLAB® R2025b

Project 2:

Problem 1:

This problem is a 4×4 matrix and need to use both Gaussian elimination and scaled partial pivoting

Given the matrix is not fully pivoted yet we need to create a for loop to move rows

to match pivots for upper triangular matrix
In order to do this similar to moving nodes we need a temp node aka Temp-row to move rows to the right pivot

that is the first loop in this section

we also use max and ratios to predetermine which rows need to be arranged.

After all that we can begin row reduction elimination to find upper triangular matrix.

once we get that from the two loops

We can do backward iteration using end element to first element

We also create a variable so "sum_of_known_parts" to help us solve for all x_i to quickly solve all x_i

and display This was interesting to do other than elimination/backward solution and no need to pivot. $\square$

Problem 2

To understand what Tri system we should solve
 $n=5$ given

$$\begin{cases} 4x_1 - x_2 = -20 \\ -x_{i-1} + 4x_i - x_{i+1} = 40 & 2 \leq i \leq n-1 \\ -x_{n-1} + 4x_n = -20 \end{cases}$$

Given first and last are given we need to
 express the middle by $n=5$ which sounds
 meaning 3 elements $2 \leq i \leq 4$

which express as

$$\begin{aligned} 4x_1 - x_2 &= -2 \\ -x_1 + 4x_2 - x_3 &= 40 \\ -x_2 + 4x_3 - x_4 &= 40 \\ -x_3 + 4x_4 - x_5 &= 40 \\ -x_4 + 4x_5 &= -20 \end{aligned}$$

Then by using Thomas algorithm we can solve this
 through Forward elimination which we go eliminate
 the first term in each equation after 1st
 to reduce to last equation to have a single term
 in last equation (e.g. $n=5$)

Then we can do back substitution to solve for
 x_i which are

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \approx \begin{pmatrix} -1.4632 \\ 14.1470 \\ 18.0513 \\ 12.8462 \\ -1.5385 \end{pmatrix}$$

now next step, how about 100 elements.

To setup we can start with our b-value of 100 ones $\times 40$ to create 100 40s. Then we matlabarray
point $b(1)$ and $b(end)$ to declare our -20 on both ends.
Then we can start setting up our Tri equations using
Matlab's ones $(n=100, 1) \times (4 \text{ or } -1)$ to create our
sub, main, and super diagonal which is our bread
and meat in our sandwich.

as we declare our x variables to display our answers.

Then we can start our forward solution
through looping all 100 elements which starts
with find the multiplier for the 2nd equation which
is current sub divided by main diagonal
Then update main diagonal and b-value with
multiplier and super diagonal and b-value
100 times.

lastly the backward solution which we can use
some of what we know from problem 1
and solve our premade diagonals and save our
results in x. Amazingly, this is similar to matrices,
just larger which helps us solve this better in
programming for 100 elements. just extra
steps. $\square$

--- MATH135A Project 2: Problem 2 ---

Solves a tridiagonal system for $n=100$ using Procedure Tri

```
disp('--- Problem 2 (n=100) ---');
n2 = 100;

% Setup the RHS vector 'b'
b = 40 * ones(n2, 1);
b(1) = -20;
b(end) = -20;

% Setup the three diagonals
% We make them size n, though a(1) and c(n) won't be used
a = -1 * ones(n2, 1); % Sub-diagonal
d = 4 * ones(n2, 1); % Main diagonal
c = -1 * ones(n2, 1); % Super-diagonal

% Pre-allocate solution vector
x2 = zeros(n2, 1);

% --- 2. Forward Elimination (Thomas Algorithm) ---
for i = 2:n2
    % Find the multiplier
    m = a(i) / d(i-1);

    % Update the main diagonal (d) and the RHS (b)
    d(i) = d(i) - m * c(i-1);
    b(i) = b(i) - m * b(i-1);
end

% --- 3. Back Substitution (Thomas Algorithm) ---
% Solve for the last x
x2(n2) = b(n2) / d(n2);

% Loop backwards from n-1 down to 1
for k = n2-1:-1:1
    % Solve for x(k)
    x2(k) = (b(k) - c(k) * x2(k+1)) / d(k);
end

% --- 4. Final Answer (Problem 2) ---
disp('Final Answer Vector x for Problem 2 (n=100):');
disp(x2);

--- Problem 2 (n=100) ---
Final Answer Vector x for Problem 2 (n=100):
-1.4359
14.2563
18.4610
19.5876
19.8895
19.9704
```

[illegible]

20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
20.0000
19.9998
19.9994
19.9979
19.9921
19.9704
19.8895
19.5876
18.4610
14.2563
-1.4359

Published with MATLAB® R2025b